



Trash Talk

Understanding Memory Management

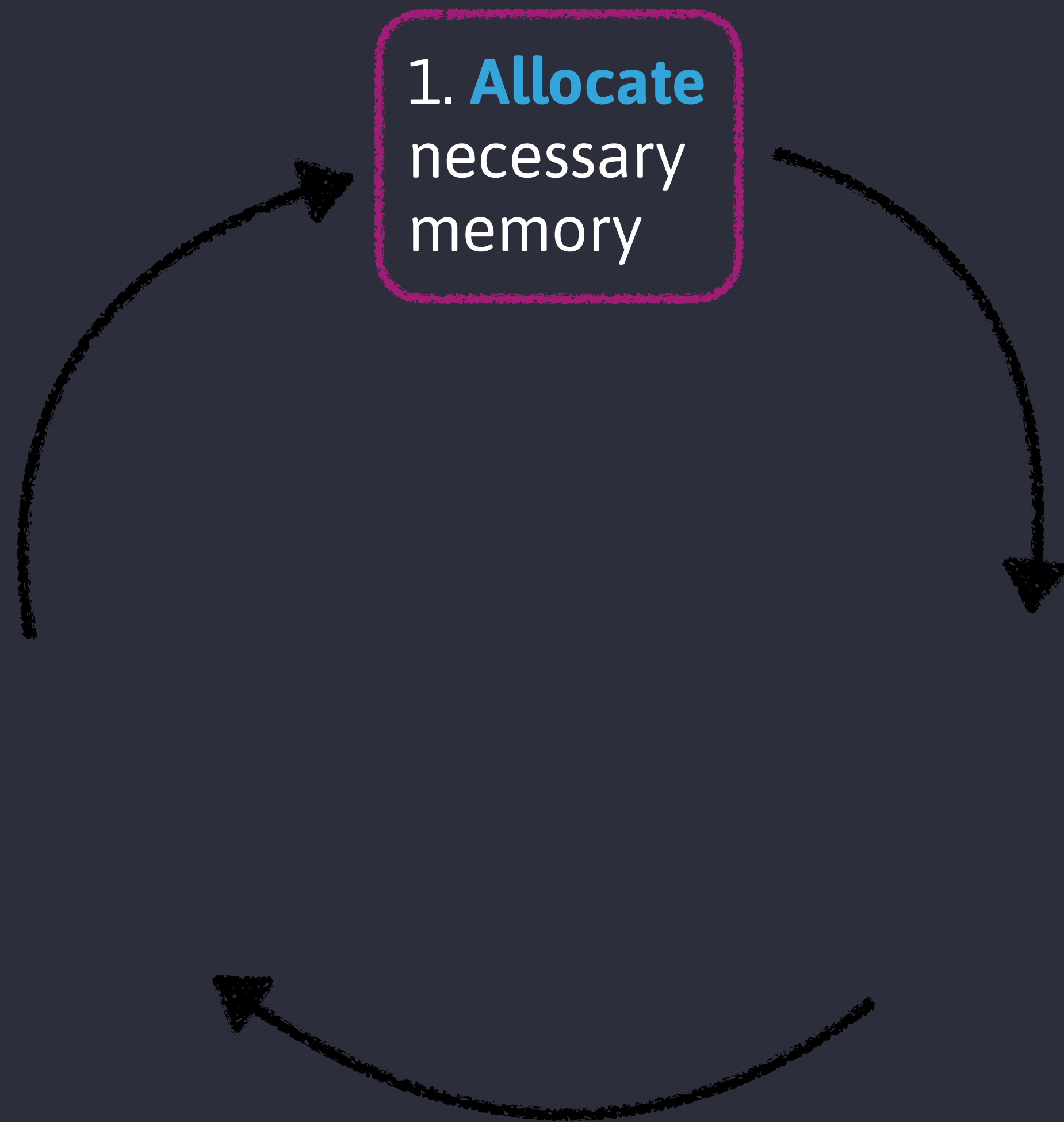
Shelley Vohr
@codebytere

Garbage Collection

A form of **automatic memory management**. Attempts to **reclaim garbage**, or memory occupied by objects that are no longer in use by the program.

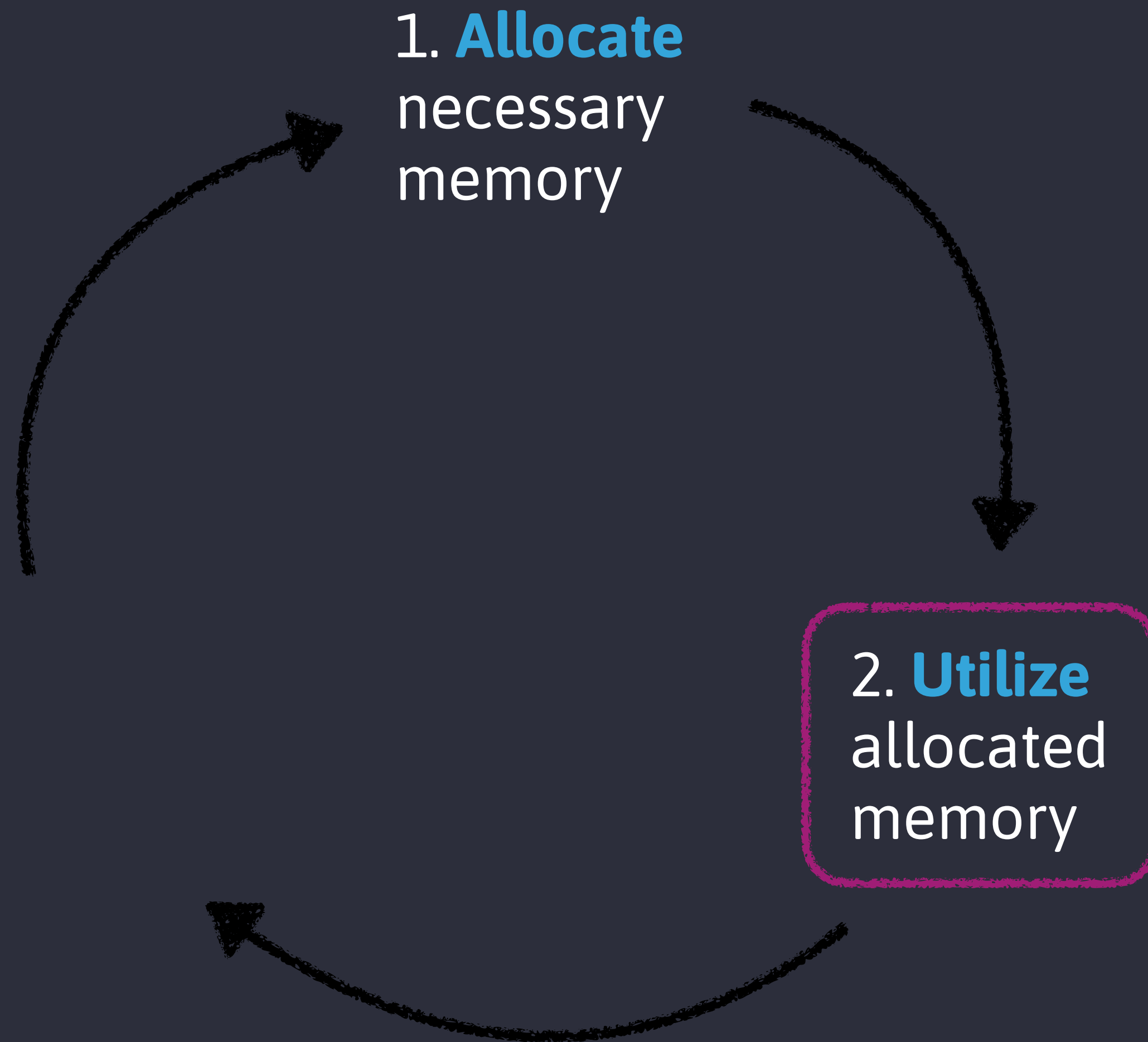
•.) You should probably care about
C/ memory management in JavaScript

Memory Lifecycle



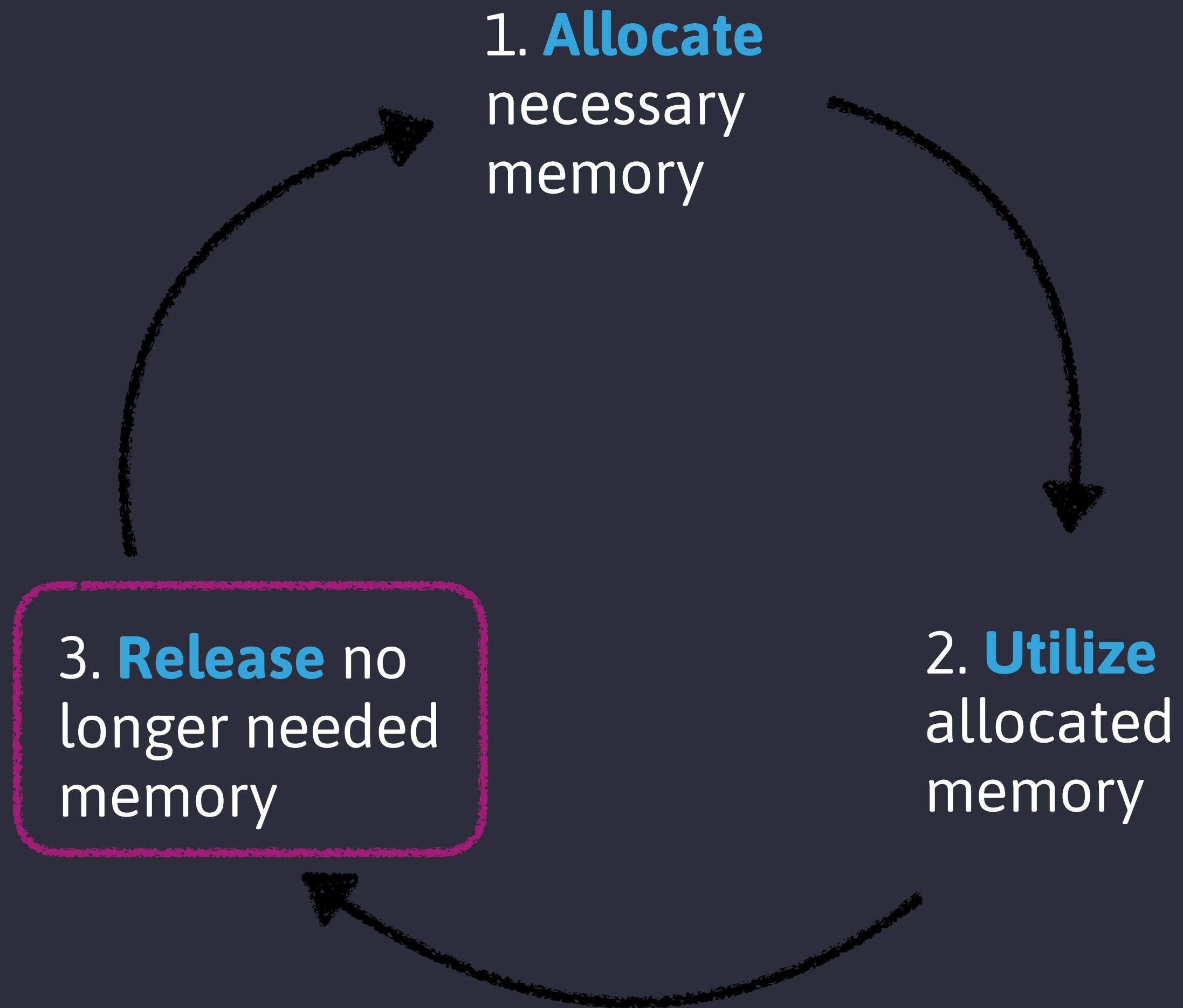
Explicit in lower-level languages like C, **implicit** in JavaScript

Memory Lifecycle



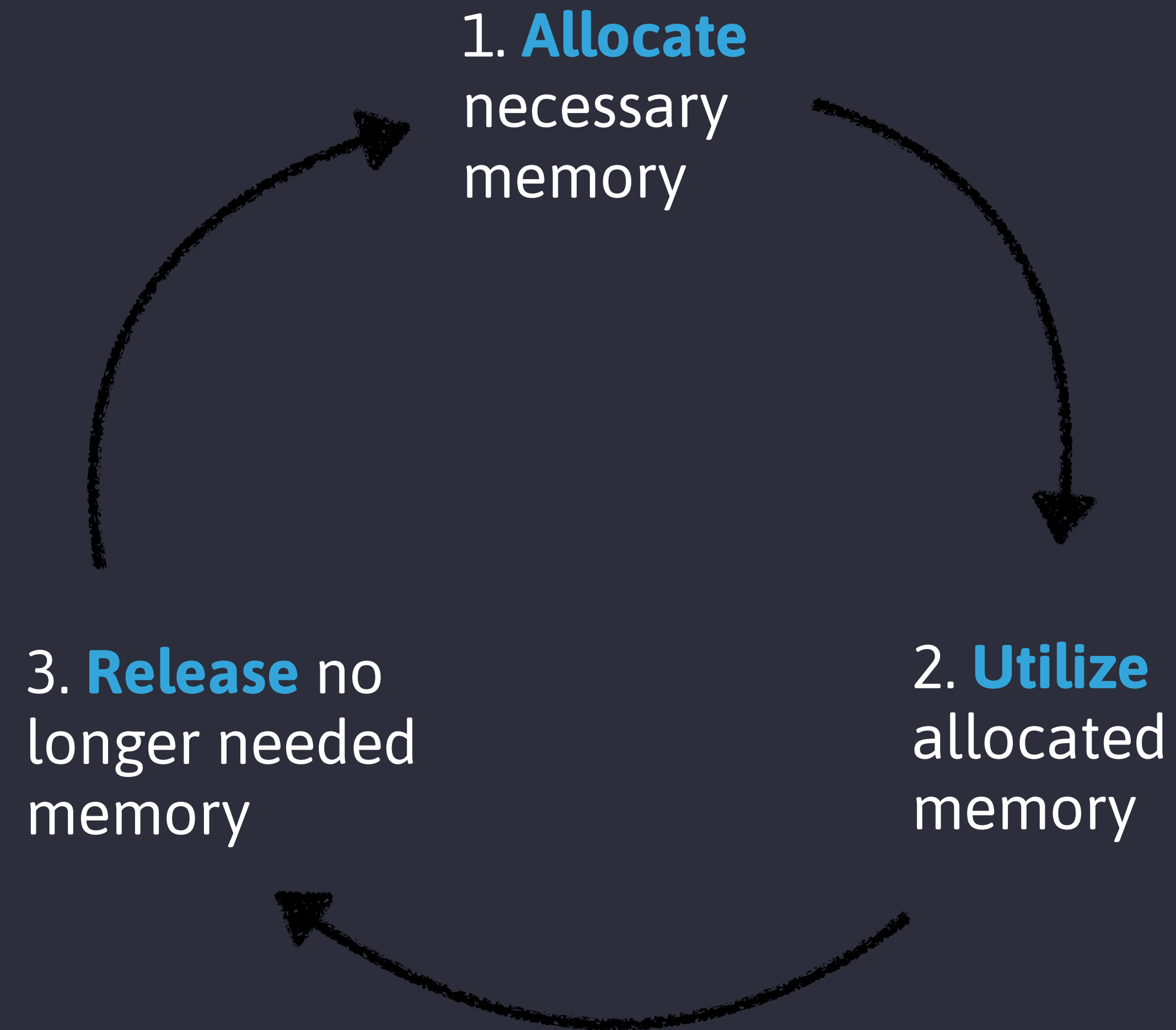
Explicit in lower-level languages like C, **implicit** in JavaScript

Memory Lifecycle



Explicit in lower-level languages like C, **implicit** in JavaScript

Memory Lifecycle



Explicit in lower-level languages like C, **implicit** in JavaScript

1. Allocate Necessary Memory

Value Initialization

```
// Allocate memory for a number  
const num = 42  
  
// Allocate memory for an object  
let name = {  
  first: 'Shelley',  
  last: 'Vohr'  
}  
  
// Allocate memory for an array  
const colors = ['red', 'green', 'blue']
```

Allocation Via Function Call

```
// Allocate memory via function call  
function greet() {  
  console.log('Hello, world!');  
}
```


2. Use Allocated Memory

Reading and writing
in allocated memory

```
let num = 42  
num += 10
```

```
const person = { firstName: 'Caroline' }  
person.firstName = 'Shelley'
```

```
let colors = ['red', 'green', 'blue']  
colors.push('yellow')
```

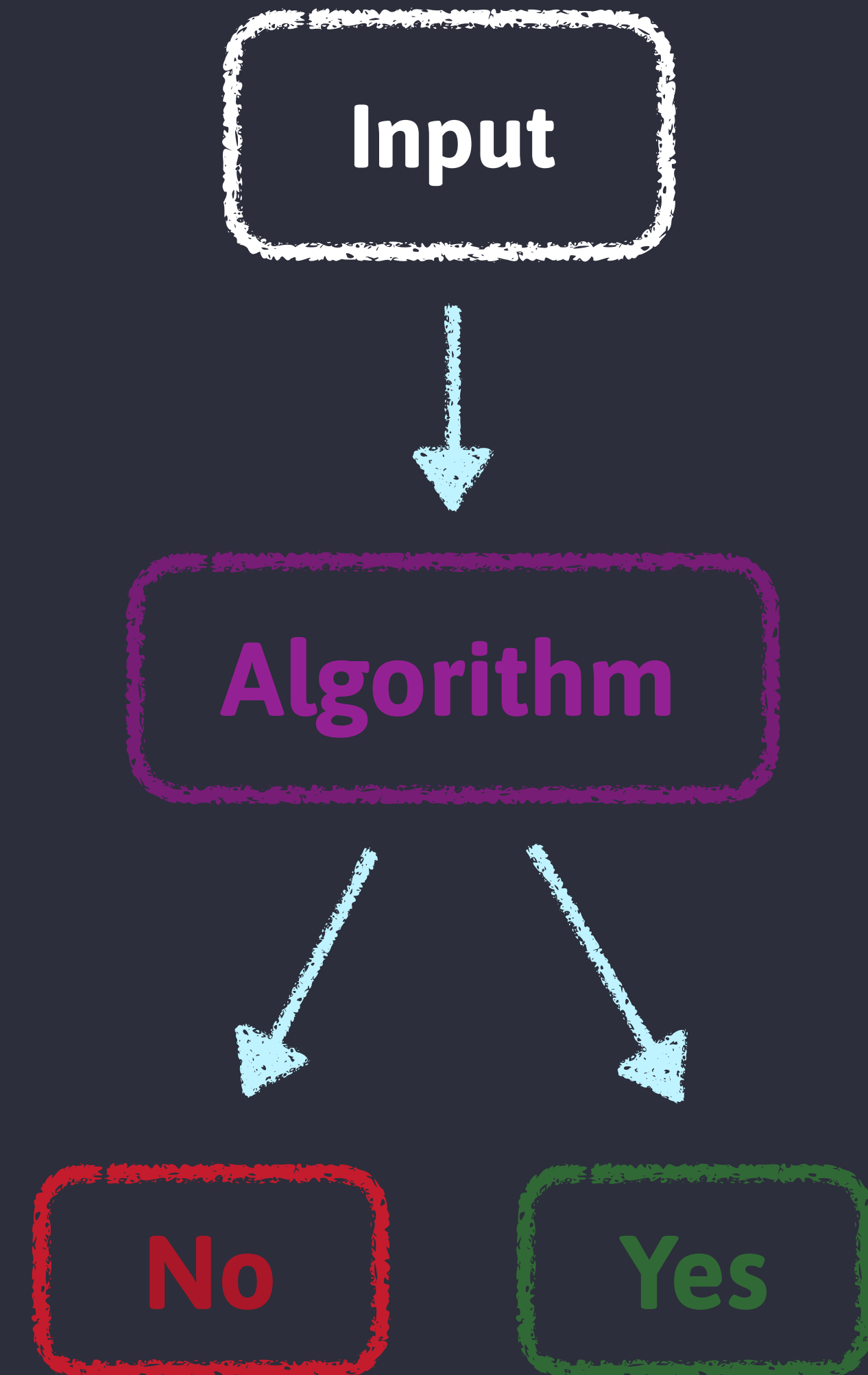

3. Release No Longer Needed Memory

Determining whether memory is reclaimable is an **undecidable problem**.

? ? ? ? ? ?
? How do we know when an ?
allocated piece of memory
is **garbage collectible**? ?
? ? ? ? ?

Decidability

A problem is **decidable** when a proven effective method for **determining membership** exists for all cases.



Reference-Counting

Whether or not an object is **still needed** → whether an object still has any **other objects referencing it**



Circular references



Performance overhead

```
// 2 objects created; nothing can be gc'd
const first = { a: { b: 'covalence' } }

const second = first

first = 1

const third = second.a

// 'first' can now be gc'd, but 'a' is
// still referenced and can't be
second = 'electron'

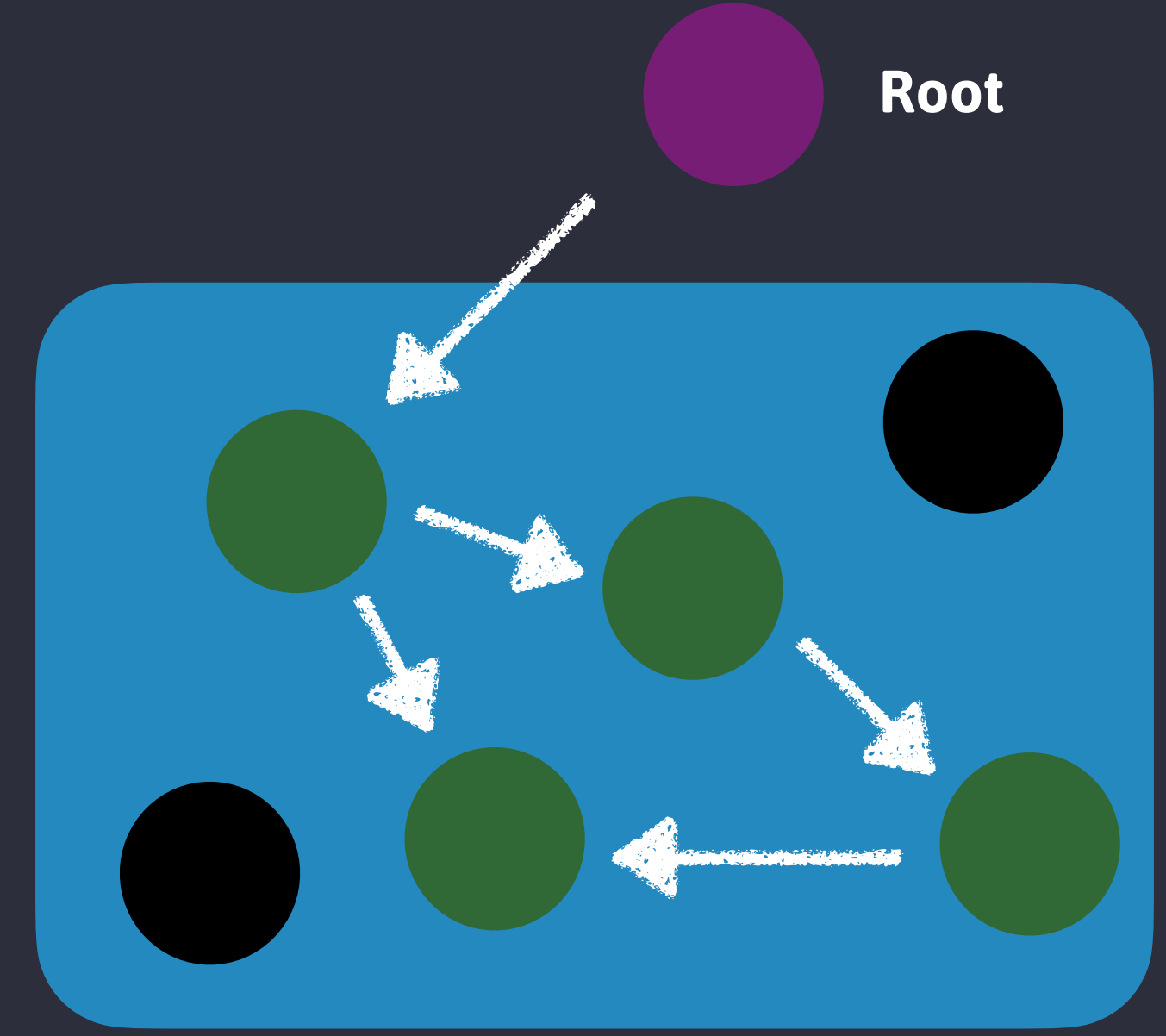
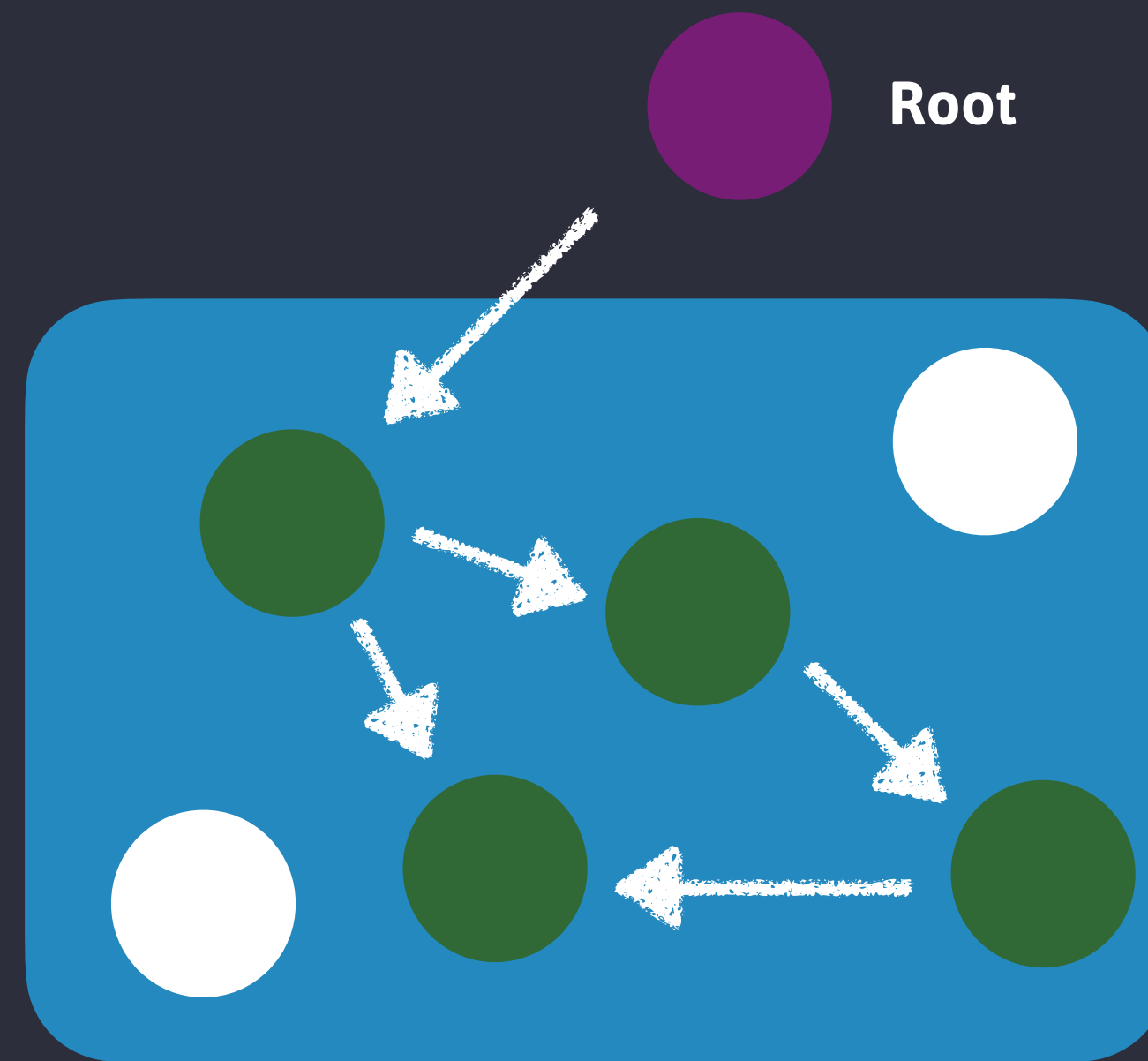
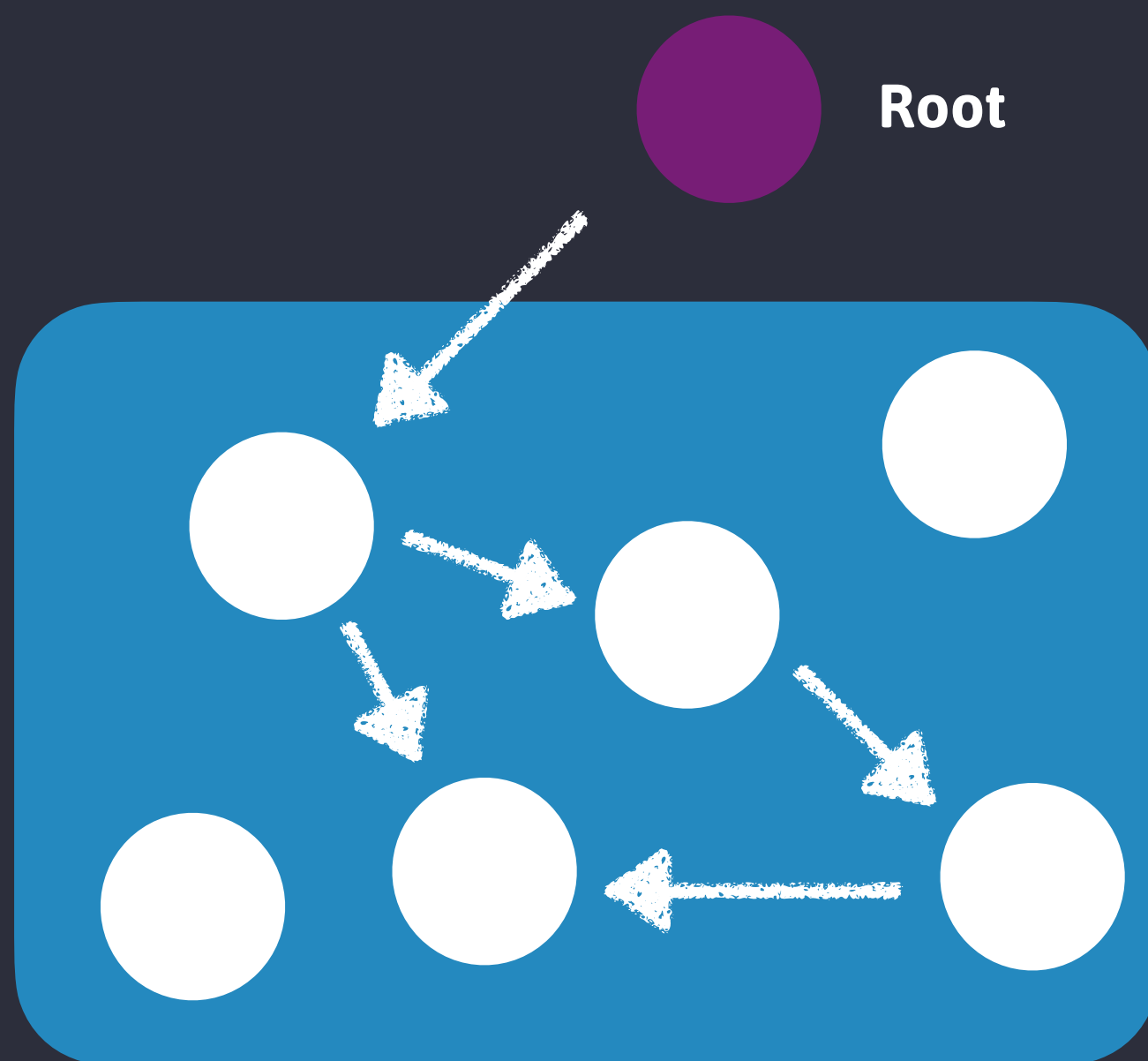
// 'a' can now be gc'd
third = null
```


Mark-and-Sweep

Whether or not an object is **still needed** → whether an object is **unreachable**

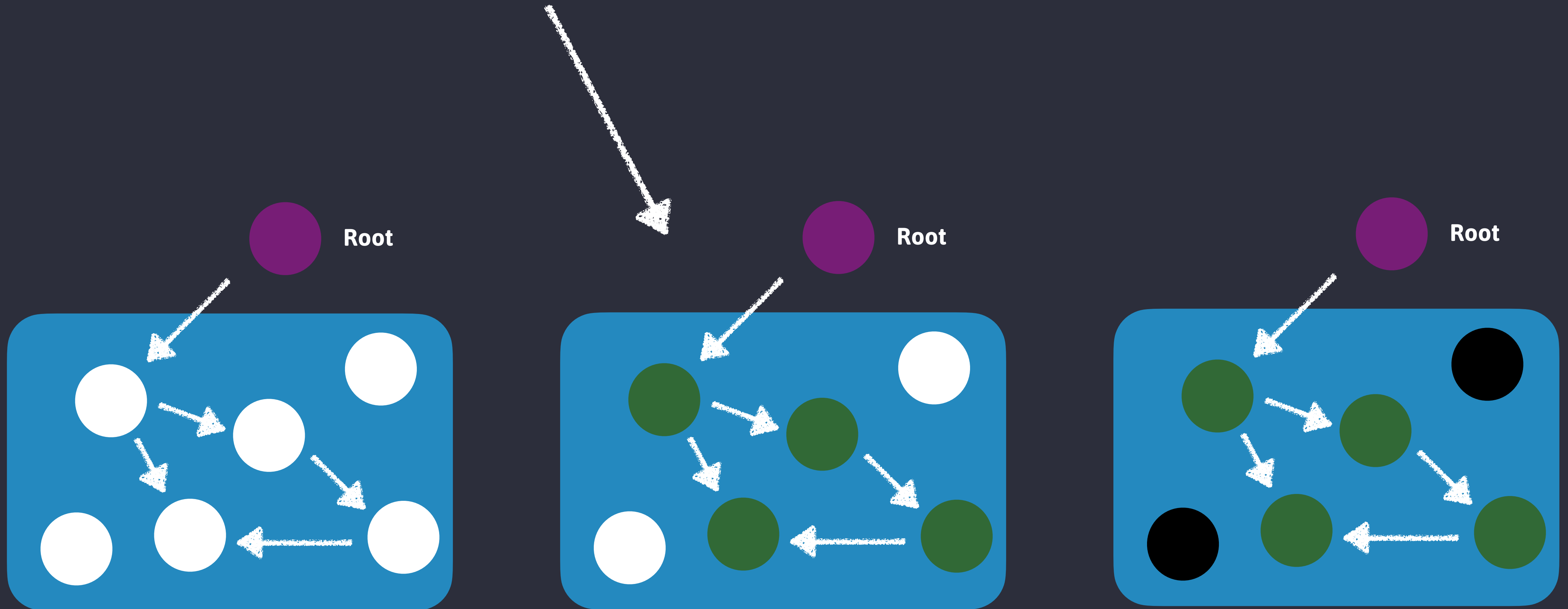


Can't manually release memory



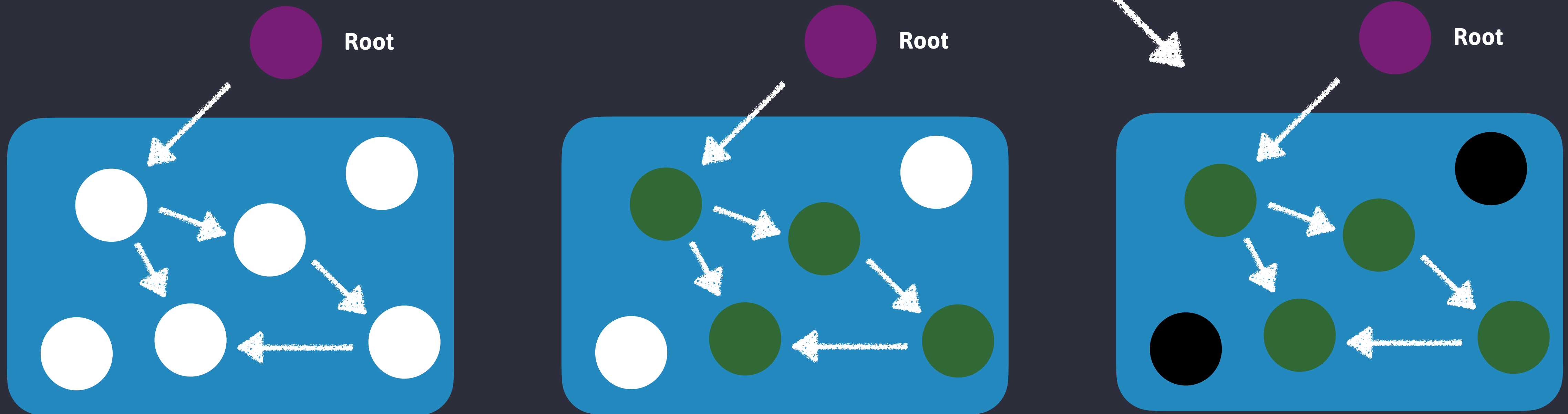
Marking

The process by which
reachable objects are found.



Sweeping

The process by which
unreachable objects are
cleared from **heap memory**.



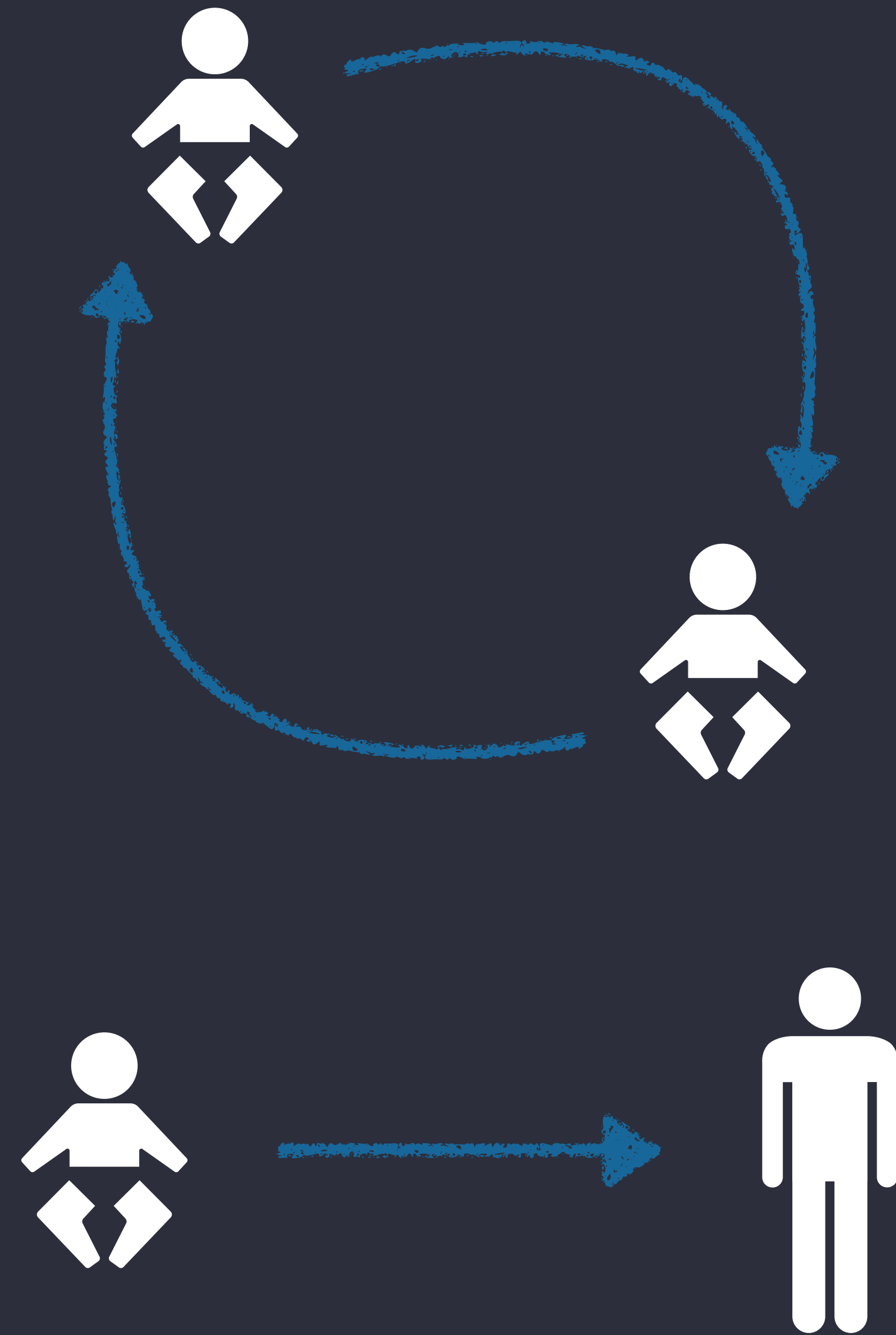


Google's open source **high-performance** JavaScript and WebAssembly **engine**, written in C++

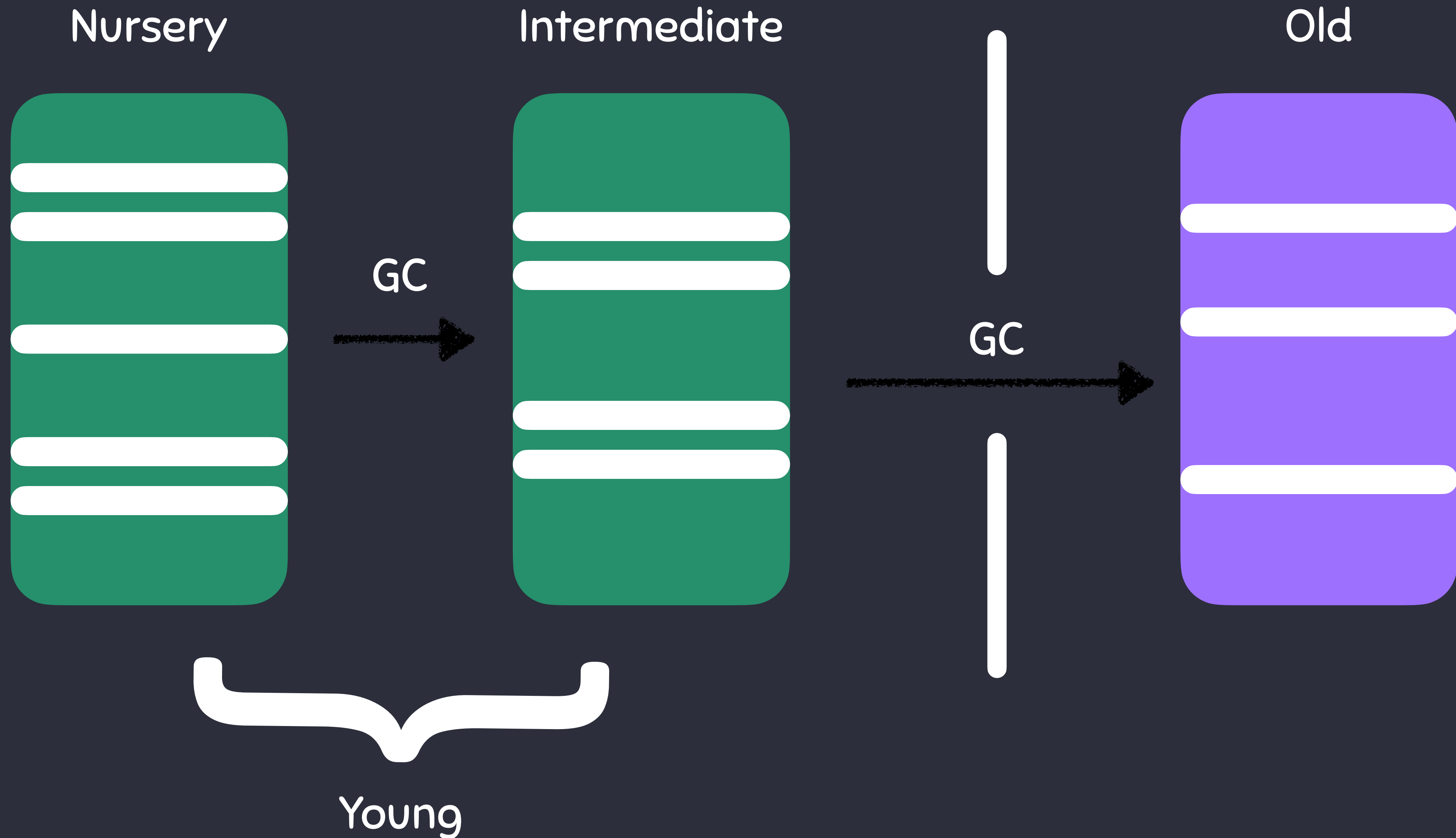
Contains a **mark-and-sweep** garbage collector.

Generational Hypothesis

- Most allocated values become **unused quickly**
- The ones that don't, usually **survive** for a long time



Heap Layout



What's a Memory Leak?

A **memory leak** is an allocated piece of memory that the garbage collector can't reclaim



Leads to **increased memory usage** and **degraded performance**

Common Leaks

Accidental Globals

Timers

Detached DOM Elements

Closures

Accidental Globals

```
function createSession(name, role) {  
  sessionData = {}  
  sessionData.user = name  
  sessionData.role = role  
  return sessionData  
}
```

Forgot to use `let` `const` or `var`!

`sessionData` will never go out of scope and get garbage collected.

Accidental Globals

```
function createSession(name, role) {  
  const sessionData = {}  
  sessionData.user = name  
  sessionData.role = role  
  return sessionData  
}
```

Used `const`!



`sessionData` goes out of scope and gets garbage collected.

Closures

```
function makeClosure () {  
  const data = new Array(1000000).fill( 'Data')  
  return function () {  
    console.log(data)  
  };  
}
```

```
const closure = createClosure()  
closure()  
closure()
```

Returned function keeps a reference to `data`

Closure is formed over `data`, preventing garbage collection

Closures

```
function makeClosure() {  
  let data = new Array(1000000).fill('Data')  
  return function () {  
    console.log(data)  
    data = null  
  };  
}
```

```
const closure = makeClosure()  
closure()  
closure()
```

Returned function releases
data

Closure is not formed over
data, allowing garbage
collection

Timers

```
function startTimer() {  
  const data = new Array(1000000).fill("Data")  
  setInterval (() => {  
    console.log("Timer running")  
  }, 1000);  
}  
  
startTimer()
```

`setInterval` keeps a reference to `data` as long as the timer is running (forever!)

Memory leakage and buildup if `startTimer` is called multiple times

Timers

```
function startTimer() {  
  const intervalId = setInterval(() => {  
    console.log("Timer running")  
  }, 1000)  
  
  setTimeout(() => {  
    clearInterval(intervalId)  
    console.log("Timer stopped")  
  }, 5000)  
}  
  
startTimer()
```

The timer is cleared when it is no longer necessary!

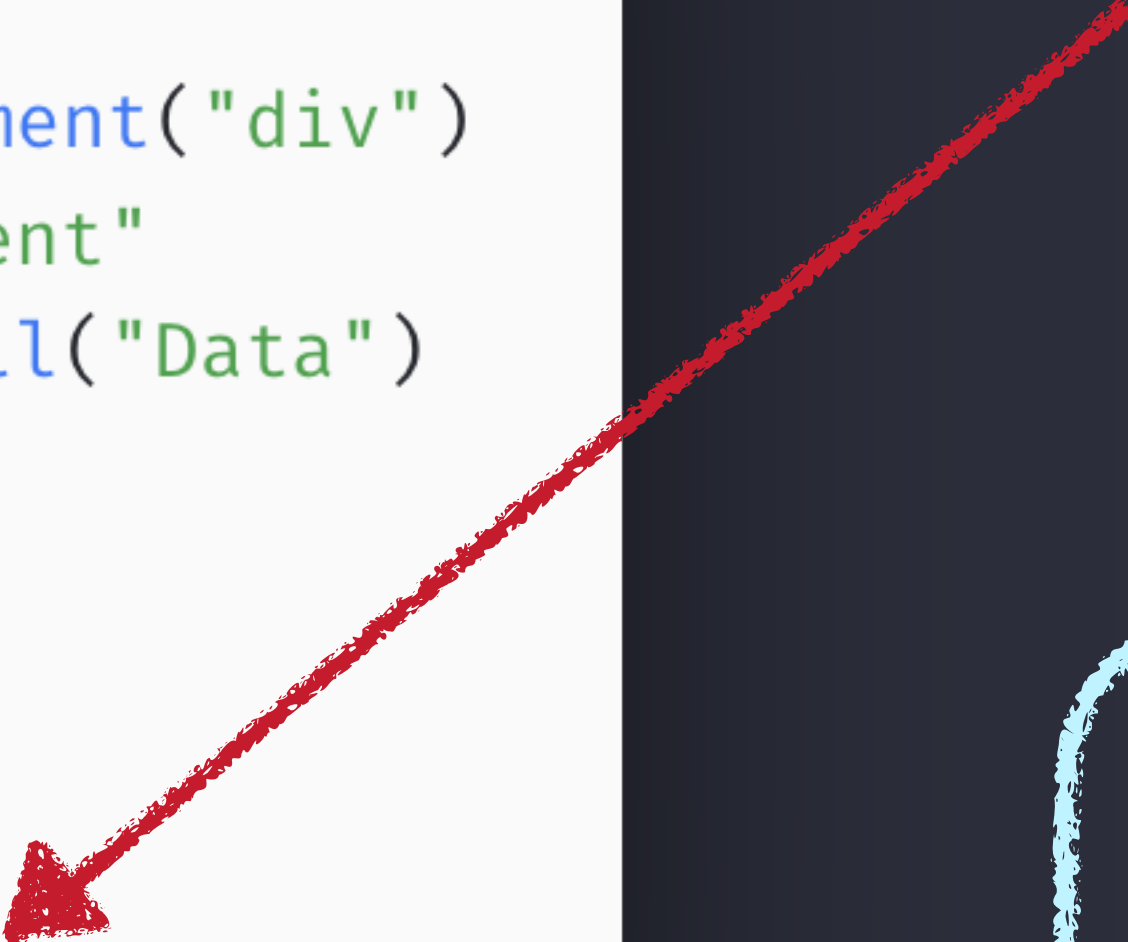
No memory leakage and buildup if `startTimer` is called multiple times!

Detached DOM Elements

```
const elementMap = new Map()

function createMappedElement () {
  const element = document.createElement("div")
  element.textContent = "Mapped Element"
  const data = new Array(1000000).fill("Data")
  elementMap.set(element, largeData)
  document.body.appendChild(element)
}

createMappedElement()
const element = document.querySelector("div")
document.body.removeChild(element)
```



`element` is later
detached from the DOM

JavaScript `Map` objects
strongly reference their
keys, `element` and `data`
remain in memory.

Detached DOM Elements

```
const elementMap = new Map()

function createMappedElement () {
  const element = document.createElement("div")
  element.textContent = "Mapped Element"
  const data = new Array(1000000).fill("Data")
  elementMap.set(element, largeData)
  document.body.appendChild(element)
}

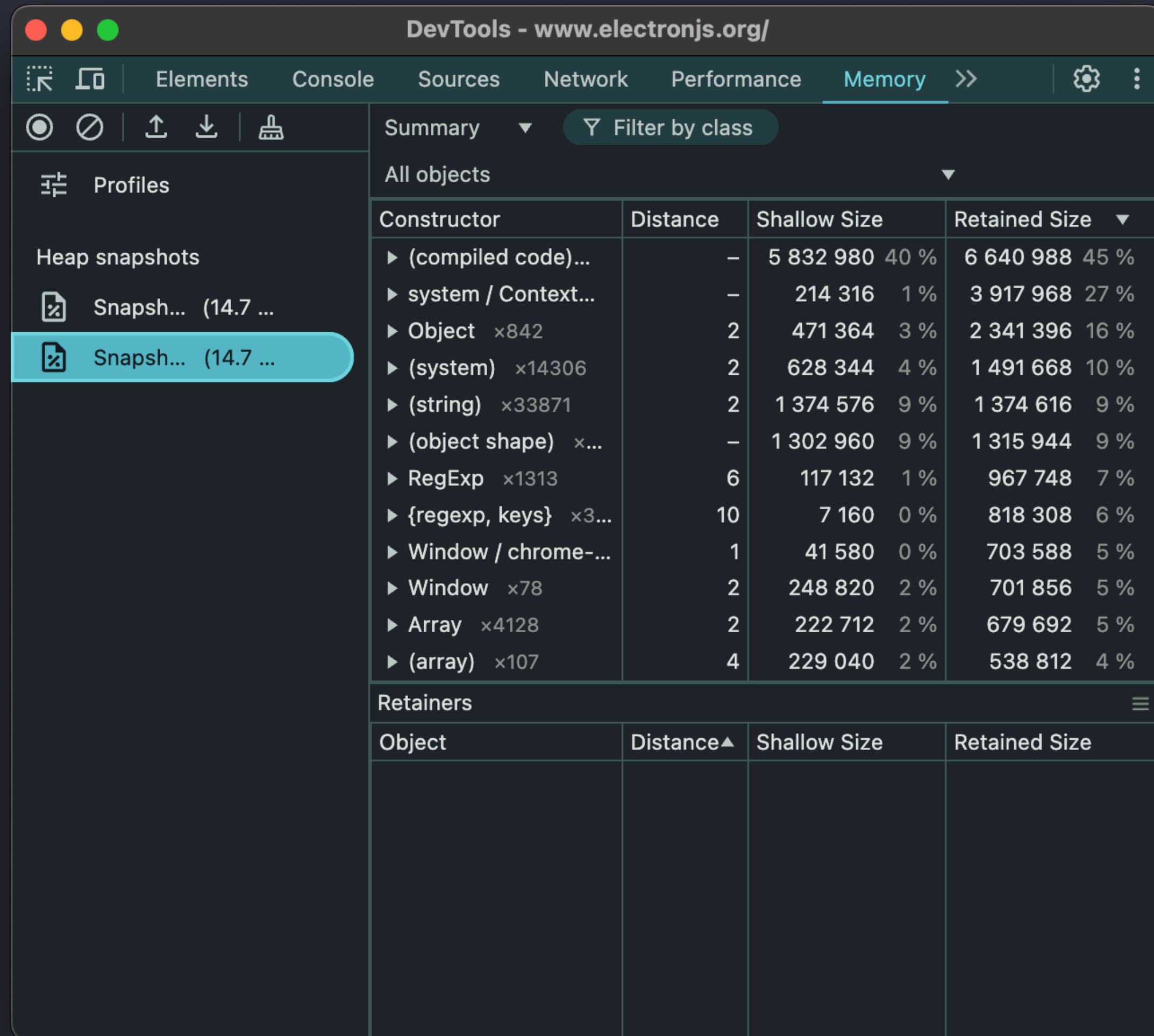
createMappedElement()
const element = document.querySelector("div")
document.body.removeChild(element)
elementMap.delete(element)
```



`element` and `data` are eligible for garbage collection

`element` is removed from `elementMap`

Heap Snapshot



The screenshot shows the Chrome DevTools interface with the Memory tab selected. The left sidebar shows 'Profiles' with 'Heap snapshots' expanded, listing two snapshots from 14.7 seconds ago. The main panel displays the 'Summary' view for 'All objects'. It includes a table with columns for Constructor, Distance, Shallow Size, and Retained Size. Below this is a 'Retainers' section with its own table.

Constructor	Distance	Shallow Size	Retained Size
▶ (compiled code)...	–	5 832 980 40 %	6 640 988 45 %
▶ system / Context...	–	214 316 1 %	3 917 968 27 %
▶ Object ×842	2	471 364 3 %	2 341 396 16 %
▶ (system) ×14306	2	628 344 4 %	1 491 668 10 %
▶ (string) ×33871	2	1 374 576 9 %	1 374 616 9 %
▶ (object shape) ×...	–	1 302 960 9 %	1 315 944 9 %
▶ RegExp ×1313	6	117 132 1 %	967 748 7 %
▶ {regexp, keys} ×3...	10	7 160 0 %	818 308 6 %
▶ Window / chrome-...	1	41 580 0 %	703 588 5 %
▶ Window ×78	2	248 820 2 %	701 856 5 %
▶ Array ×4128	2	222 712 2 %	679 692 5 %
▶ (array) ×107	4	229 040 2 %	538 812 4 %

Object	Distance▲	Shallow Size	Retained Size
--------	-----------	--------------	---------------

- Understand Memory Usage
- Diagnose Memory Leaks
- Gain Insights
- Optimize Performance

WeakMap

```
const elementMap = new WeakMap()

function createMappedElement() {
  const element = document.createElement("div")
  element.textContent = "Mapped Element"
  const data = new Array(1000000).fill("Data")
  elementMap.set(element, data)
  document.body.appendChild(element)
}

createMappedElement()

const element = document.querySelector("div")
document.body.removeChild(element)
```

- Keys are objects only
- Automatically removes keys when unreferenced
- Good for caching and private object data

WeakSet

```
let activeUsers = new WeakSet()

let user1 = { name: 'Carolyn' }
let user2 = { name: 'Caroline' }

activeUsers.add(user1)
activeUsers.add(user2)

console.log(activeUsers.has(user1)) // true
console.log(activeUsers.has(user2)) // true

// user1 can now be garbage collected
user1 = null
```

- Holds object references only
- Automatically removes keys when unreferenced
- No iteration or size checking

FinalizationRegistry

An object that lets you run **cleanup logic** when an object is garbage-collected

Avoid or use with caution!

```
const registry = new FinalizationRegistry((heldValue) => {  
  console.log(`Cleaning up resource associated with: ${heldValue}`);  
});  
  
let user = { name: "Shelley" };  
  
registry.register(user, "Shelley's resource");  
  
user = null;  
  
console.log("User object is now eligible for garbage collection.");
```



Unpredictable Timing



Performance Impact



Hard to Debug

Wrapping Up

While you can get by in JavaScript with only cursory knowledge of garbage collection, you'll write **more robust code** when you consider it actively!

- ✓ **Actively consider** UI element **lifetimes**
- ✓ **Profile** your **heap usage** regularly
- ✓ **Watch** your **globals**



THANK YOU!



github.com/codebytere



x.com/codebytere



bsky.app/profile/codebyte.re

@codebytere